

OWASP crAPI Security Assessment

Prepared By:

Atharva Joshi

Target:

OWASP crAPI

Classification:

Internal Training Project

crAPI Penetration Test Report

Executive Summary

This report presents the results of a security assessment performed against the **crAPI (Completely Ridiculous API)** application. The objective of the assessment was to identify security weaknesses affecting the confidentiality, integrity, and availability of the application through manual penetration testing.

The assessment focused on identifying common web and API security vulnerabilities, including authentication flaws, authorization issues, insecure object references, mass assignment, information disclosure, and business logic weaknesses. Testing was performed using industry-standard methodologies aligned with the OWASP Web Security Testing Guide (WSTG) and OWASP API Security Top 10.

During the assessment, several security issues were identified, including Broken Object Level Authorization (BOLA), Broken Function Level Authorization (BFLA), Mass Assignment, Sensitive Information Disclosure, and insecure password reset functionality. These vulnerabilities could allow an attacker to access unauthorized resources, modify privileged attributes, and disclose sensitive application data.

Overall, the application demonstrates multiple authorization and input validation weaknesses that could significantly impact the security of user accounts and business data if deployed in a production environment.

Assessment Objectives

The objectives of this penetration test were to:

- Identify vulnerabilities affecting the application's security.
- Evaluate authentication and authorization mechanisms.
- Assess API endpoints for common OWASP API Security risks.
- Verify access controls for user and administrative functionality.
- Identify insecure configurations and information disclosure issues.
- Provide remediation recommendations to improve the application's security posture.

Testing Methodology

The assessment was conducted using a manual penetration testing approach following industry best practices. Testing included reconnaissance, authentication testing, authorization testing, input validation, parameter manipulation, business logic testing, and API security assessment.

The following activities were performed during the engagement:

- Application reconnaissance and endpoint enumeration.
- Analysis of HTTP requests and responses.
- JWT authentication analysis.
- Parameter tampering.
- Broken Access Control testing.
- Broken Object Level Authorization (BOLA) testing.
- Broken Function Level Authorization (BFLA) testing.
- Mass Assignment testing.
- Sensitive Information Disclosure testing.
- Password Reset workflow assessment.
- Business Logic testing.

Assessment Scope

The assessment covered the exposed functionality of the crAPI application, including:

- User Registration
 - User Authentication
 - User Profile Management
 - Password Reset Functionality
 - Vehicle Management
 - Order Management
 - Report Management
 - Administrative Functionality
 - REST API Endpoints
 - JWT-based Authentication
-

Tools Used

Tool	Purpose
Burp Suite Professional	Proxying, request interception, API testing
Postman	API testing
JWT.io	JWT analysis and decoding
Chrome Developer Tools	Client-side analysis
Kali Linux	Testing environment

Broken Object Level Authorization (BOLA) – Unauthorized Access to Other Users' Reports

Severity :

High

Description :

The application failed to enforce object-level authorization when accessing reports. The API relied solely on the user-supplied report_id to retrieve report information without verifying whether the authenticated user was authorized to access the requested resource.

By modifying the report_id parameter, an authenticated user could retrieve reports belonging to other users.

Affected Endpoint :

GET /api/v2/reports/{report_id}

(Replace with the actual endpoint if it differs.)

Steps to Reproduce :

1. Log in as a normal user.
2. Intercept the request used to retrieve a report.
3. Identify the report_id value.
4. Modify the report_id to another valid identifier.
5. Forward the request.
6. Observe that the server returns another user's report instead of denying access.

This vulnerability violates authorization controls and may serve as a stepping stone for further attacks against other users.

Root Cause

The application validates that the requester is authenticated but does not verify ownership of the requested resource. Authorization decisions are based solely on the supplied `report_id`, allowing direct object reference manipulation.

Remediation

- Enforce object-level authorization on every request.
- Verify that the authenticated user owns or is permitted to access the requested report.
- Never rely solely on client-supplied object identifiers for authorization decisions.
- Implement centralized authorization checks across all API endpoints.
- Log and monitor repeated access attempts to unauthorized resource identifiers.

Broken Object Level Authorization (BOLA) – Unauthorized Download of Other Users' Reports

Severity

High

Description

The application failed to enforce object-level authorization when processing report download requests. The download functionality relied solely on a user-controllable filename parameter to determine which report to return, without verifying whether the authenticated user was authorized to access the requested file.

By modifying the filename parameter, an authenticated user could download reports belonging to other users.

Affected Endpoint

GET /api/v2/report/download?filename=report_1.pdf

(Replace the endpoint with the actual one from Burp if different.)

Steps to Reproduce

1. Log in as a standard user.
2. Navigate to the report download functionality.
3. Intercept the request using Burp Suite.
4. Identify the filename parameter.
5. Modify the value from your own report to another valid report name (for example, report_5.pdf).
6. Forward the request.

7. Observe that the server returns another user's report instead of denying access.

Evidence

Original Request

GET /workshop/api/mechanic/download_report?filename=report_6
HTTP/1.1 Authorization: Bearer <JWT>

Modified Request

GET /workshop/api/mechanic/download_report?filename=report_5
HTTP/1.1 Authorization: Bearer <JWT>

Result

The server successfully returned the report belonging to another user without performing an ownership check.

The screenshot shows a network request and response in a web browser's developer tools. The request is a GET to `/workshop/api/mechanic/download_report?filename=report_5` with an Authorization header. The response is a PDF document titled "ReportLab Generated PDF document" with a status of 200 OK. The response headers include `Server: openresty/1.27.1.2`, `Date: Fri, 12 Jun 2026 13:14:12 GMT`, `Content-Type: application/octet-stream`, `Content-Length: 3684`, `Connection: keep-alive`, `Content-Disposition: inline; filename="report_5"`, `Allow: GET, HEAD, OPTIONS`, `Vary: origin, Cookie`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: same-origin`, and `Cross-Origin-Opener-Policy: same-origin`. The response body is a PDF document.

Impact

An authenticated attacker can download reports belonging to other users, leading to unauthorized disclosure of potentially sensitive information. Depending on the contents of these reports, this could

expose personal information, confidential business data, or other protected records.

The vulnerability compromises data confidentiality and may facilitate further attacks by allowing attackers to gather information about other users.

Root Cause

The application authorizes access based solely on the user-supplied filename parameter and does not validate whether the requested report belongs to the authenticated user before serving the file.

Remediation

- Implement server-side ownership validation before returning any downloadable file.
- Avoid exposing predictable filenames or identifiers.
- Associate report downloads with authenticated user permissions rather than client-supplied filenames.
- Return an authorization error (such as HTTP 403 Forbidden) when unauthorized resources are requested.
- Log repeated attempts to access unauthorized files.

Mass Assignment Leading to Product Price Manipulation

Severity

High

OWASP Mapping

- **OWASP API Top 10 2023:** API3 – Broken Object Property Level Authorization (Mass Assignment)
 - **OWASP Top 10 2021:** A04 – Insecure Design
-

Description

The application allows authenticated users to modify sensitive product attributes through the update product endpoint. By intercepting the request in Burp Suite and modifying the price parameter, it was possible to assign arbitrary values, including negative numbers.

The server accepted the user-controlled value without validating whether the authenticated user was authorized to modify this property or whether the supplied value was within an acceptable range.

Steps to Reproduce

1. Log in as a normal user.
2. Navigate to the product update functionality.
3. Capture the request using Burp Suite.
4. Modify the price parameter.

Example:

5. `{"id":1,"name":"Seat","price":"-10.00","image_url":"images/seat.svg"}`
6. Forward the request.
7. Observe that the server accepts the request and updates the product price.

Proof of Concept

Original Request

```
{
  "price": "10.00"
```

Modified Request

```
{
  "price": -"10.00"
}
```

Result

- Product price successfully changed.
- Negative pricing was reflected by the application.

```
POST /workshop/api/shop/products?limit=30&offset=0 HTTP/1.1
Host: localhost:8443
Cookie: chat_session_id=96217e23-43bd-44c3-a430-37e5b1781ec6
Sec-Ch-Ua-Platform: "Linux"
Authorization: Bearer eyJhbGciOiJSUzI1NiIsImR5cyI6ZmVudC5ib20iLCJpcyYXQiOiE3ODQ0eWJkOTYtYTUwLjY0MGEtMTg3NDkxNDEwIiwiaWF0IjoiZS16bWVzZXI1fQ..bT6B8R4VsIUvPbdaH2yiuvOztEpbb8jg3voXvMtV9oXvWynN0ZHEXGVZchuBeDHGr1XXiSDOGAJUFXosg56O1qXKF8LsnBa8mfKThOEZK2-hkyivJO-ketOIQDeqvJnilgan8YFfhSmidi4HB0BUl17KOLX7mfZi-SArCmAOLrGXgbkKiHt.dSIaGtAlPyFzUqMoFR0nFgzTT3vyv8ZSSSHdDeiq604auexOmKV-_Fop3b6BSR4UhpeDGMwyIxPk07JLSrnUwp9oDYdwHAPmSeq-KxxkdVBNaGLmZMaPG0XqnjnSTJmLUbOtKsyIDV-tGYBsN3HWFOyrw4w_gmaipyg
Accept-Language: en-US,en;q=0.9
Sec-Ch-Ua: "NotIA;Brand",v="8", "Chromium",v="138"
Content-Type: application/json
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36
Accept: */*
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://localhost:8443/shop
Accept-Encoding: gzip, deflate, br
Priority: u=1,i
Connection: keep-alive
Content-Length: 69

{
  "id":1,
  "name":"Seat",
  "price":"-10.00",
  "image_url":"images/seat.svg"
}
```

Impact

An attacker can manipulate product prices without authorization, potentially resulting in:

- Free purchases
- Financial losses
- Corruption of business data
- Abuse of discounts and pricing logic
- Fraudulent transactions

If exploited in production, this could significantly impact the application's integrity and revenue.

Root Cause

The application trusts client-supplied input for sensitive business fields and fails to:

- Restrict which properties users can modify (mass assignment).
- Validate that pricing values fall within acceptable business rules.
- Enforce server-side authorization for sensitive attributes.

Remediation

- Implement allowlisting for properties that users are permitted to update.
- Ignore sensitive fields such as price unless the requester has administrative privileges.
- Perform server-side validation to reject invalid values (for example, negative prices).
- Enforce role-based access control before allowing updates to business-critical attributes.

Business Logic Vulnerability Leading to Unauthorized Account Credit

Severity

Critical

OWASP Mapping

- **OWASP API Top 10 2023:** API6 – Unrestricted Access to Sensitive Business Flows
 - **OWASP Top 10 2021:** A04 – Insecure Design
-

Description

The application fails to validate the quantity of products during checkout. By modifying the quantity parameter to a negative value, the application processed the order successfully and credited money to the attacker's account instead of charging for the purchase.

This indicates that the application performs financial calculations using client-supplied values without enforcing server-side validation.

Steps to Reproduce

1. Log in as a normal user.
2. Add a product to the shopping cart.
3. Intercept the checkout/update request using Burp Suite.
4. Modify the product quantity.

Example:

```
{  
  "product_id": 5,  
  "quantity": -5  
}
```

5. Forward the request.
 6. Complete the purchase.
 7. Observe that the application credits money to the user's account instead of deducting the purchase amount.
-

Proof of Concept

Original Request

```
{  
  "product_id":2,  
  "quantity": 100  
}
```

Modified Request

```
{  
  "product_id":2,  
  "quantity": -100  
}
```

Result

- Request accepted.
- Order processed successfully.
- User balance increased instead of decreasing.

- | Request | | Response | |
|---|-----|---|-----|
| Pretty | Raw | Pretty | Raw |
| <pre> 1 POST /workshop/api/shop/orders HTTP/1.1 2 Host: localhost:8443 3 Cookie: chat_session_id=96217e23-43bd-44c3-a430-37e5b1781ec6 4 Content-Length: 36 5 Sec-Ch-Ua-Platform: "Linux" 6 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cGE6bnVhbGkiOiJkaGk5bWVpbGpjb20iLCJpYXN0IjE3ODUyNjktOTYzOTYyInV4cCI6MThtc3NDE5NiwiYm95 ZSI6InVzZXIiLCJ0b284R4Vsiu8VesVpbdaH2yiuV0ZtEpb8Jg3vo0xVmtv9oxWyyNzOTHEXGVzchuBeDHGr1XXi90GAlUfxOsg60 0l9xKFL9snBaBNfKTh9ERzk2-kHyiv0-keTOIQDegvJnllgan8YFMnSedi4HBoBUL17K0L7xf21-5ArCaAOLrGx6bkkIhtdStAot alPyfzUqMoFRokFgzTl3yvyBZSSShDeiq604aueOmKv- Fop3b6B5R4UHpeDGMmyIxFk07JL5RmUwp9oDydwHARnSeq-KxKdVBNAa LmZMaPGDQxnjn5TJmLubOTKsyIDV-tGYBsN3HWFOYr4w_gmralpyg 7 Accept-Language: en-US,en;q=0.9 8 Sec-Ch-Ua: ("Not)A;Brand";v="8", "Chromium";v="138" 9 Content-Type: application/json 10 Sec-Ch-Ua-Mobile: 70 11 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36 12 Accept: */* 13 Origin: https://localhost:8443 14 Sec-Fetch-Site: same-origin 15 Sec-Fetch-Mode: cors 16 Sec-Fetch-Dest: empty 17 Referer: https://localhost:8443/shop 18 Accept-Encoding: gzip, deflate, br 19 Priority: u=1, i 20 Connection: keep-alive 21 22 { "product_id":2, "quantity":100 } </pre> | | <pre> 1 HTTP/1.1 200 OK 2 Server: openresty/1.27.1.2 3 Date: Fri, 12 Jun 2026 13:25:47 GMT 4 Content-Type: application/json 5 Connection: keep-alive 6 Allow: GET, POST, PUT, HEAD, OPTIONS 7 Vary: origin, Cookie 8 access-control-allow-origin: * 9 X-Frame-Options: DENY 10 X-Content-Type-Options: nosniff 11 Referrer-Policy: same-origin 12 Cross-Origin-Opener-Policy: same-origin 13 Content-Length: 62 14 15 { "id":11, "message":"Order sent successfully.", "credit":1070.0 } </pre> | |

- Performs business calculations without enforcing server-side business rules.

Remediation

- Validate that product quantity is a positive integer greater than zero.
- Reject negative, zero, or excessively large quantities.
- Perform all pricing and balance calculations exclusively on the server.
- Add business-rule validation before processing transactions.
- Implement automated tests for edge cases involving quantities and pricing.

Broken Object Level Authorization (BOLA) in Order Retrieval

Severity

High

Description

The application fails to verify whether the authenticated user is authorized to access the requested order. By modifying the order ID in the request, it was possible to retrieve order details belonging to other users.

The API only checks whether the requester is authenticated and does not verify ownership of the requested resource.

Steps to Reproduce

1. Log in as a normal user.
2. Navigate to your order history.
3. Intercept the request using Burp Suite.

Example:

GET /workshop/api/shop/orders/4 HTTP/1.1

Authorization: Bearer <JWT>

4. Change the order ID.

GET /workshop/api/shop/orders/5 HTTP/1.1

Authorization: Bearer <JWT>

5. Forward the request.
 6. Observe that the response contains another user's order information.
-

- Access other users' orders.
- View personal and purchase information.
- Enumerate all order records.
- Gather sensitive business data.
- Violate customer privacy.

Root Cause

The application validates authentication but fails to verify that the requested order belongs to the authenticated user.

Authorization checks are performed only at the endpoint level and not on individual objects.

Remediation

- Enforce ownership checks for every object returned by the API.
- Ensure each order is associated with the authenticated user before returning data.
- Return **403 Forbidden** when unauthorized access is attempted.
- Implement centralized object-level authorization across all API endpoints.

Broken User Authentication

Description

The application provides a Forgot Password feature that sends a One-Time Password (OTP) to the registered email address for password recovery.

The latest v3 OTP verification endpoint implements rate limiting to prevent brute-force attacks. However, by modifying the API version from v3 to v2, the same OTP verification functionality was accessible without any rate limiting.

Using a valid victim email address, an attacker can repeatedly guess OTP values against the legacy endpoint until the correct OTP is found. This allows the attacker to reset the victim's password and gain full access to their account.

Steps to Reproduce

1. Obtain a valid user email address.
2. Click Forgot Password on the login page.
3. Submit the victim's email address.
4. Observe that the application uses a v3 API endpoint to verify the OTP.
5. Enter several incorrect OTPs and observe that rate limiting is enforced.
6. Intercept the request in Burp Suite.
7. Modify the endpoint from:

POST identity/api/auth/v3/check-otp

to:

POST identity/api/auth/v2/check-otp

8. Use Burp Intruder to brute-force the OTP.

9. Observe that no rate limiting is applied.

After discovering the correct OTP, complete the password reset and log in as the victim.

Proof of Concept

Protected Endpoint

- **POST identity/api/auth/v3/check-otp**
- **Rate limiting enforced.**

Legacy Endpoint

POST identity/api/auth/v2/check-otp

- **Unlimited OTP attempts allowed.**

Result

- **OTP successfully brute-forced.**
- **Password reset completed.**
- **Full account takeover achieved.**

Request	Payload	Status code ^	Error	Timeout	Length
3791	3790	200	☐	☐	500
0		500	☐	☐	519
1	0000	500	☐	☐	519
2	0001	500	☐	☐	519
3	0002	500	☐	☐	519
4	0003	500	☐	☐	519
5	0004	500	☐	☐	519
6	0005	500	☐	☐	519
7	0006	500	☐	☐	519
8	0007	500	☐	☐	519
9	0008	500	☐	☐	519
10	0009	500	☐	☐	519
11	0010	500	☐	☐	519
12	0011	500	☐	☐	519
13	0012	500	☐	☐	519

Request	Response
	Pretty Raw Hex Render <pre> 6 Vary: Origin 7 Vary: Access-Control-Request-Method 8 Vary: Access-Control-Request-Headers 9 Access-Control-Allow-Origin: * 10 X-Content-Type-Options: nosniff 11 X-XSS-Protection: 1; mode=block 12 Cache-Control: no-cache, no-store, max-age=0, must-revalidate 13 Pragma: no-cache 14 Expires: 0 15 X-Frame-Options: DENY 16 Content-Length: 39 17 18 { "message": "OTP verified", "status": 200 } </pre>

Impact

Successful exploitation allows an attacker to:

- Reset another user's password.
- Gain unauthorized access to victim accounts.
- Access sensitive personal information.
- Perform actions as the compromised user.
- Completely take over affected accounts.

Root Cause

The application exposes multiple API versions with inconsistent security controls.

Although the latest API version protects OTP verification with rate limiting, the legacy version lacks these protections, allowing attackers to bypass authentication safeguards.

Remediation

- **Apply identical security controls across all API versions.**
- **Remove or disable deprecated API versions.**
- **Implement centralized rate limiting that cannot be bypassed by changing API versions.**
- **Restrict the number of OTP verification attempts.**
- **Temporarily lock the Forgot Password process after repeated failures.**
- **Monitor and alert on excessive OTP verification attempts.**

Broken Function Level Authorization

Severity

High

Description

The application fails to enforce authorization checks on administrative HTTP methods. A normal authenticated user was able to invoke a privileged **DELETE** operation against the video management endpoint and successfully delete videos without possessing administrative privileges.

The API exposes functionality intended for administrators but does not verify the user's role before executing the request.

Steps to Reproduce

1. Log in as a normal user.
2. Navigate to the video management section.
3. Intercept the request.

Example:

PUT /identity/api/v2/user/videos/61

4. Change the HTTP method from **PUT** to **DELETE**.

DELETE /identity/api/v2/user/videos/61

5. Forward the request.
 6. Observe that the video is deleted successfully despite using a non-administrative account.
-

Proof of Concept

Original Request

PUT /identity/api/v2/user/videos/61

Modified Request

DELETE /identity/api/v2/user/videos/61

Result

The video was successfully deleted without requiring administrator privileges.



Challenge 7 Complete

Impact

An attacker can:

- Delete arbitrary videos.
- Modify application content.
- Cause loss of business data.
- Abuse administrative functionality.

Root Cause

The application authorizes users at the authentication level but fails to verify whether the authenticated user is permitted to execute privileged operations such as DELETE.

Remediation

- Enforce role-based authorization for every API endpoint and HTTP method.
- Restrict DELETE operations to authorized administrative users.
- Validate user roles on the server before performing sensitive actions.

Broken Function Level Authorization

Severity

High

OWASP Mapping

- **OWASP API Top 10 2023:** API5 – Broken Function Level Authorization
 - **OWASP Top 10 2021:** A01 – Broken Access Control
-

Description

The order API fails to restrict privileged update operations. A normal user was able to convert a read-only **GET** request into a **PUT** request and update the status of arbitrary orders by supplying another order ID.

This functionality should only be accessible to privileged users such as administrators or order management personnel.

Steps to Reproduce

1. Log in as a normal user.
2. Open an order.
3. Intercept the request.

Original request:

GET /workshop/api/shop/orders/4

4. Change the method to **PUT**.
5. Modify the order ID to another user's order.

Example:

PUT /workshop/api/shop/orders/5

6. Modify the request body.

Example:

```
{  
  "status": "returned"  
}
```

7. Forward the request.

8. Observe that another user's order status is updated successfully.

Proof of Concept

Original Request

GET /workshop/api/shop/orders/4

Modified Request

PUT /workshop/api/shop/orders/5

Body:

```
{  
  "status": "returned"  
}
```

Result

The status of another user's order was successfully changed.

Remediation

- Restrict order management operations to authorized roles.
- Enforce server-side authorization for all state-changing HTTP methods.
- Validate both user role and resource ownership before updating order information.